

Dust Reproduction

Dust (Dahal et al., 2026) trains a transformer language model without a backward pass. It adds noise to activations at every token and turns the loss changes that follow into a gradient estimate. The paper reports that this matches backpropagation (backprop) at pretraining. We ran the official release, added the AdamW baseline it lacks, and tested Dust on models that backprop had already trained.

CONTENTS

What was run	Cost
Does the release reproduce the paper?	Claims we did not rerun
Against AdamW	Outside criticism
Can tuning close the gap?	Limits of this reproduction
Continuing a backprop checkpoint	References

SUMMARY

- The release reproduces the paper. Its Dust runs with stochastic gradient descent (SGD) and our reconstruction of Dust under Adam all land within 0.05 of the published test losses ([Table 1](#)).
 - The paper's main comparison trains backprop with SGD at a constant learning rate. With AdamW, backprop is about 1.0 nat better at 10M tokens, and no Dust run with SGD comes near it ([Table 2](#)).
 - Dust under Adam, which the paper reports only at 1M tokens, is level with backprop AdamW there at the largest population. At 10M tokens it tracks backprop for the first hundred steps and then falls steadily behind, ending about 0.5 nat above it at a population of 16,384 ([Figure 2](#)). Changing the learning rate or β_1 does not close the gap ([Table 4](#)).
 - Started from a model that backprop has already trained, Dust still lowers the loss, but only at a learning rate below the one tuned for training from scratch. At a population of 16,384 it reaches 73% and 85% of backprop's improvement from our two checkpoints ([Table 6](#)). Nothing suggests that it finds directions backprop misses.
 - The cost is large. At 1M tokens the run with a population of 16,384 used roughly 600 times the graphics processing unit (GPU) time of backprop ([Table 7](#)).
 - What this reproduction does not cover, including populations above 16,384, is stated in the scope note below and in the limits at the end.
-

SCOPE AND DISCLAIMER

This is an independent reproduction, not affiliated with the paper's authors. It covers one model size (37.7M parameters), training from scratch on up to 10M tokens, populations up to 16,384 and, unless a table says otherwise, one seed per Dust run. Dust under Adam is our reconstruction from the paper's appendix. The report was drafted with Claude Opus 5.5 from the run records; any errors are ours.

We did not test whether a larger population closes the gap to AdamW. At 10M tokens, quadrupling the population from 4,096 to 16,384 narrows the gap to backprop with AdamW from 0.62 to 0.50 nat, at four times the compute (Table 2). If every further quadrupling helped as much as that one, closing the gap would take about 4 more, a population in the millions. That is an extrapolation from two points with one seed each, and the earlier quadruplings helped more, so read it as the order of magnitude and no more.

What was run

For a linear layer with output $\mathbf{y}_t$ at token t , Dust adds noise $\sigma \mathbf{a}_t$ with $\mathbf{a}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, reruns the forward pass, and rewards each token's noise with the loss reduction r_t it caused. Averaged over K independent draws,

$$\hat{\mathbf{g}}_t = -\frac{1}{K\sigma} \sum_{i=1}^K r_t^{(i)} \mathbf{a}_t^{(i)}$$

estimates the error at the layer's output, and its outer product with the layer's cached input is the weight gradient. K is the population. Attention queries, keys and values are scored against the estimated error of the attention output instead of the loss. The derivation is below for readers who want it.

Derivation: why noise times loss change estimates the gradient

What is estimated. For $\mathbf{y}_t = \mathbf{W}\mathbf{x}_t$, backprop gives $\partial L / \partial \mathbf{W} = \sum_t \delta_t \mathbf{x}_t^\top$ with $\delta_t = \partial L / \partial \mathbf{y}_t$. The input $\mathbf{x}_t$ is known from the forward pass, so only δ_t needs the backward pass. Dust estimates δ_t without it.

The estimator. The noise goes on the layer's output, not on its weights: $\mathbf{y}_s \rightarrow \mathbf{y}_s + \sigma \mathbf{a}_s$ at every token s , with $\mathbf{W}$ unchanged. A first-order Taylor expansion of the loss in these outputs gives the perturbed loss $L + \sigma \sum_s \delta_s^\top \mathbf{a}_s$, where $\delta_s = \partial L / \partial \mathbf{y}_s$ is the quantity defined above at token s ; the index s runs over all tokens and t is the one being estimated. The reward is therefore $r \approx -\sigma \sum_s \delta_s^\top \mathbf{a}_s$. Multiplying by one token's noise and averaging,

$$-\frac{1}{\sigma} \mathbb{E}[r \mathbf{a}_t] = \mathbb{E}[\mathbf{a}_t \mathbf{a}_t^\top] \delta_t + \sum_{s \neq t} \mathbb{E}[\mathbf{a}_t \mathbf{a}_s^\top] \delta_s = \delta_t,$$

because $\mathbb{E}[\mathbf{a}_t \mathbf{a}_t^\top] = \mathbf{I}$ and the noise at different tokens is independent with zero mean. Replacing the expectation by an average over K draws gives $\hat{\mathbf{g}}_t$.

Rewards per token. The total loss also moves because of the noise at every other token, and that part of the reward has nothing to do with $\mathbf{a}_t$. Over many draws it averages to zero, so it does not bias the estimate. In any single draw it is a large random number multiplying $\mathbf{a}_t$: for a sequence of T tokens its variance is on the order of T times that of the signal, so about T times as many draws would be needed. Dust therefore rewards a token with the loss reduction $\mathbf{c}_s$ at individual positions, not with the change of the total loss. Let ℓ_s be the next-token cross-entropy at position s and $\ell_s^{(i)}$ its value under draw i . For the n draws evaluated together in one batched forward pass, $\mathbf{c}_s^{(i)} = \frac{1}{n} \sum_{j=1}^n \ell_s^{(j)} - \ell_s^{(i)}$: how much lower the loss at s is under this draw than under the average draw. The reward of the noise at token t is then

$$r_t = \sum_{s \geq t} \gamma^{s-t} \mathbf{c}_s.$$

Under causal attention a perturbation at t cannot change the loss at earlier tokens, so dropping them only removes variance. The decay γ trades bias for variance. With $\gamma = 1$ the estimate is unbiased for backprop's δ_t to first order. With $\gamma = 0$ it keeps only the token's own loss and estimates $\partial \ell_t / \partial \mathbf{y}_t$, leaving out the later tokens that read this activation through attention. The paper uses $\gamma = 0$ for layers scored by the loss and about 0.98 for keys and values. The baseline in $\mathbf{c}_s$ is the mean over draws and not the unperturbed loss because noise shifts the loss by a second-order amount that is about the same in every draw, typically upward, and that common offset would multiply $\mathbf{a}_t$ as pure noise. A baseline that does not depend on $\mathbf{a}_t$ leaves the expectation unchanged, since $\mathbb{E}[\mathbf{a}_t] = \mathbf{0}$. This mean includes the draw itself, which shrinks the estimate by the known factor $(n-1)/n$ without changing its direction. The release uses $n = 2$ to 16 depending on the population.

Why the population must be large. One draw, $\mathbf{a} (\mathbf{a}^\top \delta)$, has the right mean, but $\mathbf{a}$ has d dimensions and only the one along δ carries signal. After K draws the cosine with the true gradient is roughly $1/\sqrt{1 + c/K}$ with c of the order of d , plus interference from the other perturbed tokens and layers. The paper fits $\mathbf{c}_{\max}/\sqrt{1 + c/K}$ to its measured cosines.

Virtual population. Adding $\sigma \mathbf{a}_t$ to the output at token t is the same as using the weights $\mathbf{W} + \sigma \mathbf{a}_t \mathbf{x}_t^\top / \|\mathbf{x}_t\|^2$ at that token alone, a rank-one weight perturbation. Each token has its own $\mathbf{a}_t$, so one forward pass evaluates as many weight perturbations as there are tokens in the batch, where weight-space evolution strategies evaluate one.

Attention internals. Noise on queries, keys and values moves the loss too little to read. Dust first estimates the error $\hat{\mathbf{g}}_s^o$ at the attention output as above, then recomputes only that block's attention under the noise and scores the change $\Delta \mathbf{o}_s$ of its output with $\mathbf{c}_s = -\langle \hat{\mathbf{g}}_s^o, \Delta \mathbf{o}_s \rangle$, a linear stand-in for the loss reduction.

Our reading, not the paper's: with $\gamma = 0$ the estimate leaves out the effect of an activation on later tokens, so it targets a truncated gradient rather than backprop's. That may be one reason the fitted ceilings $c_{\max}$ stay below one for most layers. And since the estimate gets noisier relative to the signal as training proceeds, it fits what we see below: the gap to backprop widens with steps, and a trained checkpoint needs a smaller learning rate.

Paper	Dahal, Mandal, Gülbahar and Vegesna, Dust: Pretraining Transformers Without Backpropagation , Q Labs, October 2026.
Code	The official release at commit 2fdb01b, unmodified for every SGD result. It is a minimal implementation: each draw reruns the whole forward pass, where the paper reruns only the blocks after the perturbed one.
Model	A decoder-only transformer with 8 blocks, width 512 and 37.7M parameters: rotary positions, normalized queries and keys, a squared-ReLU feed-forward layer, value embeddings and an untied output head.
Data	FineWeb with a 4,096-token byte-pair encoding (BPE) vocabulary, in batches of 8 sequences of 2,048 tokens: 61 steps for 1M tokens and 610 for 10M. Validation and test are 544 held-out sequences each.
Metric	Test loss at the checkpoint with the lowest validation loss, as in the paper.
Added here	Backprop with AdamW at the paper's Adam settings (Appendix F); Dust under Adam, rebuilt from Appendices E and F because the release has only SGD; and continuation runs from backprop checkpoints.
Seeds	Three for backprop, one (42) for every Dust run.

The release covers 1M and 10M tokens at populations from 256 to 16,384 with SGD. It leaves out the population of 64, the 100k- and 20M-token budgets, the EGGROLL baseline, the Adam runs, the model-size study and the gradient-cosine measurements, so none of those is checked here except Adam at 1M tokens.

Does the release reproduce the paper?

Yes. Every cell the release covers lands on the published number, and so does backprop with SGD ([Table 1](#), [Figure 1](#)). The Adam rows also test our reconstruction: the appendix gives the number of draws per layer family under Adam but not their split across blocks, so we rescaled the SGD split. The four published points at 1M tokens are matched.

Table 1. Test loss against the published values. Published values are means over three seeds (SGD) or five (Adam); ours are one seed for Dust. The paper has no AdamW or Adam run at 10M tokens.

Run	Ours	Paper	Difference
1M TOKENS (61 STEPS)			
Backprop (SGD), 3 seeds	5.959	5.959	0.000
Backprop (AdamW), 3 seeds	5.364	5.361	0.003
Dust (SGD), 256	6.054	6.067	-0.013
Dust (SGD), 1k	5.933	5.934	-0.001
Dust (SGD), 4k	5.878	5.911	-0.033
Dust (SGD), 16k	5.862	5.916	-0.054
Dust (Adam), 256	5.843	5.836	0.007
Dust (Adam), 1k	5.603	5.594	0.009
Dust (Adam), 4k	5.499	5.500	-0.001
Dust (Adam), 16k	5.382	5.402	-0.020
10M TOKENS (610 STEPS)			
Backprop (SGD), 3 seeds	4.989	4.989	0.000
Backprop (AdamW), 3 seeds	3.939	–	–
Dust (SGD), 256	5.230	5.231	-0.001
Dust (SGD), 1k	5.118	5.111	0.007
Dust (SGD), 4k	5.070	5.072	-0.002
Dust (SGD), 16k	5.047	5.049	-0.002
Dust (Adam), 256	5.171	–	–
Dust (Adam), 1k	4.854	–	–
Dust (Adam), 4k	4.560	–	–
Dust (Adam), 16k	4.436	–	–

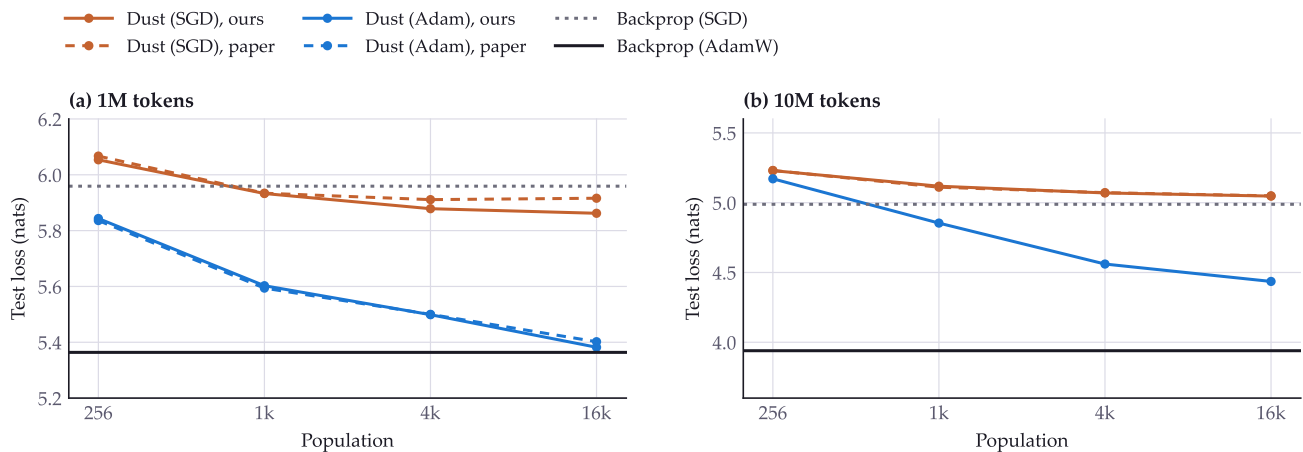


Figure 1. Test loss against population. Solid lines with points are our runs and dashed lines the published means. The horizontal lines are backprop with SGD (dotted) and with AdamW (solid).

Against AdamW

Within one optimizer the paper's claim holds. With SGD, Dust is level with backprop from a population of about a thousand at 1M tokens and close behind it at 10M. But SGD at a constant learning rate is a weak way to train this model. Backprop with AdamW is far better at both budgets, and its lead over SGD grows with tokens (Table 2). The paper's own Adam experiment stops at 1M tokens, where a whole run is 61 steps.

Table 2. Test loss of every method and its distance above backprop with AdamW. Backprop rows are means of three seeds, Dust rows one seed.

Method	1M: test loss	1M: above AdamW	10M: test loss	10M: above AdamW
Backprop (SGD)	5.959	0.60	4.989	1.05
Backprop (AdamW)	5.364	0.00	3.939	0.00
Dust (SGD), 256	6.054	0.69	5.230	1.29
Dust (SGD), 1k	5.933	0.57	5.118	1.18
Dust (SGD), 4k	5.878	0.51	5.070	1.13
Dust (SGD), 16k	5.862	0.50	5.047	1.11
Dust (Adam), 256	5.843	0.48	5.171	1.23
Dust (Adam), 1k	5.603	0.24	4.854	0.91
Dust (Adam), 4k	5.499	0.13	4.560	0.62
Dust (Adam), 16k	5.382	0.02	4.436	0.50

At 10M tokens Dust under Adam is the better Dust variant from a population of 1k, and it beats backprop with SGD. Against backprop with AdamW the answer depends on when one looks (Figure 2). For the first hundred steps the largest population is nearly on top of backprop. From a population of 1k the gap then widens until the end of training; at 256 Dust starts slowly, so the gap

narrows before it widens again (Table 3). A larger population delays the divergence and does not remove it, which agrees with the paper's own remark that the population needed to match backprop grows with tokens.

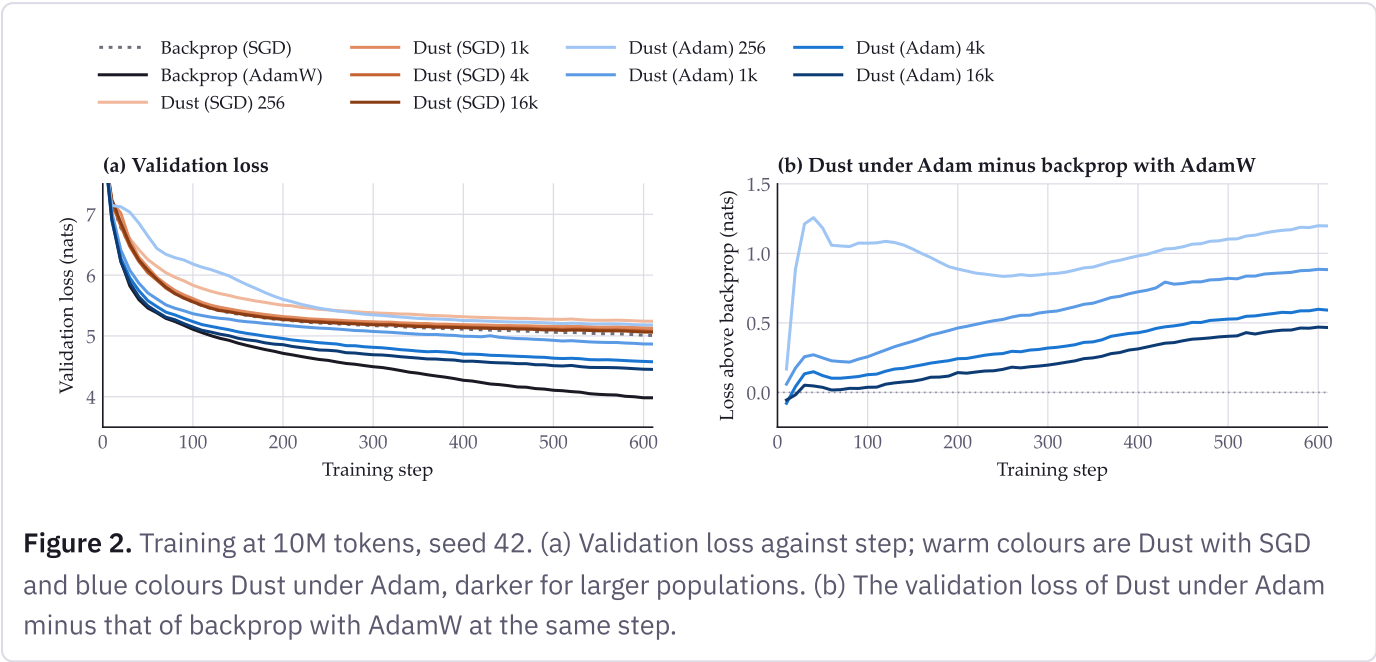


Table 3. Validation loss of backprop with AdamW at 10M tokens, and how far above it Dust under Adam is at the same step (seed 42).

Step	100	200	300	400	500	610
Backprop (AdamW), validation loss	5.11	4.71	4.49	4.27	4.11	3.98
Dust (Adam), 256	1.07	0.89	0.85	0.98	1.10	1.20
Dust (Adam), 1k	0.26	0.46	0.58	0.72	0.82	0.88
Dust (Adam), 4k	0.13	0.24	0.32	0.43	0.53	0.59
Dust (Adam), 16k	0.04	0.14	0.20	0.31	0.40	0.47

Can tuning close the gap?

Not along the two axes we tried. Dust's Adam settings come from the paper's 61-step runs, so at a population of 1k we varied the two that most plausibly depend on run length, the learning rate and β_1 . Halving the learning rate helps slightly and every other change is worse (Table 4). Backprop's learning rate is bracketed at both budgets, with the paper's value best.

Table 4. Test loss under different optimizer settings, seed 42.

Setting	Test loss
BACKPROP (ADAMW), MATRIX LEARNING RATE; SEED 42	
1M tokens, learning rate 0.001	5.555
1M tokens, learning rate 0.002	5.372
1M tokens, learning rate 0.004	5.453
10M tokens, learning rate 0.0005	4.160
10M tokens, learning rate 0.001	3.987
10M tokens, learning rate 0.002	3.977
10M tokens, learning rate 0.004	4.154
DUST (ADAM), POPULATION 1K, 10M TOKENS; SEED 42	
Appendix settings: learning rate 0.0015, β_1 0.7	4.854
Learning rate halved (0.00075)	4.832
Learning rate doubled (0.003)	4.987
β_1 0.9	4.976
β_1 0.95	5.003

Seeds do not change the picture. We repeated one Dust cell with three seeds; they agree to within 0.01, far less than the distance to backprop with AdamW ([Table 5](#)).

Table 5. Test loss by seed at 10M tokens.

Method	Seed 42	Seed 43	Seed 44	Mean
Backprop (AdamW)	3.977	3.901	3.940	3.939
Dust (Adam), 4k	4.560	4.554	4.568	4.561

We did not vary how draws are split across layers, the noise scales or the credit decay. The paper tuned these for SGD in its largest 10M-token cell and reports that only a slower decay and a shift of draws toward attention paid off there.

Continuing a backprop checkpoint

The paper argues that backprop explores the loss landscape poorly. If so, a zeroth-order method should be able to improve a model that backprop has trained, a test proposed by You Jiacheng. We trained two checkpoints with backprop and AdamW, on 10M and on 100M tokens, and continued each for 200 steps on tokens that neither the checkpoint nor the held-out sets contain. All runs but one start from a fresh optimizer state, so backprop and Dust are treated alike; the remaining run keeps backprop's AdamW state as the uninterrupted reference.

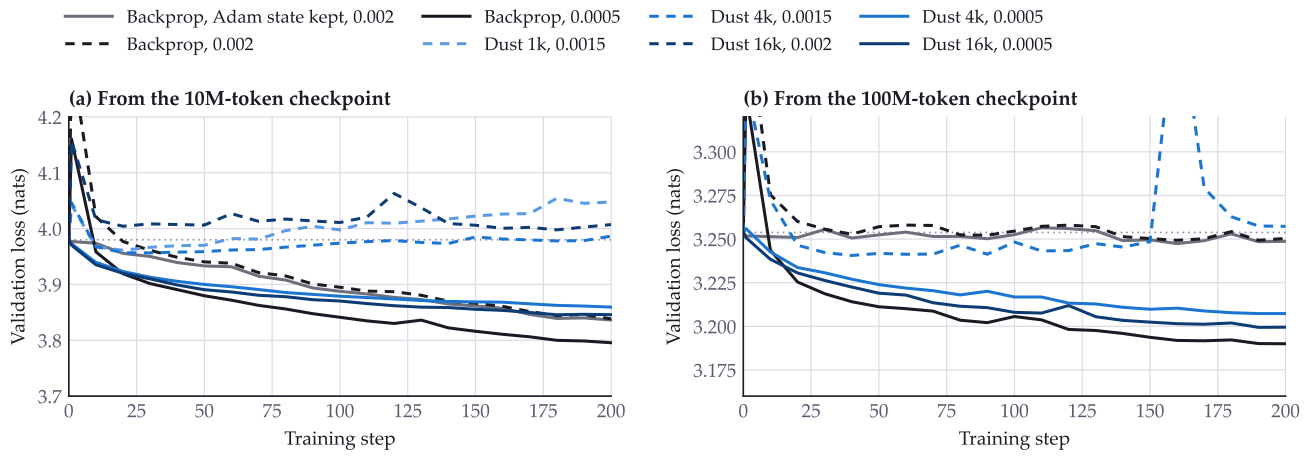


Figure 3. Validation loss over 200 continuation steps. The dotted grey line is each checkpoint's starting loss. Legend entries give the method and the matrix learning rate; solid lines use 0.0005 and dashed lines the rates tuned for training from scratch. The first step with a fresh optimizer state raises the loss above the plotted range.

Table 6. Validation loss during continuation. The last column is the drop of a Dust run from the starting loss by step 200, as a share of the drop of backprop at the same learning rate of 0.0005. Dust's learning rate is that of its matrices; its embedding and scalar rates scale with it as in the appendix.

Method and matrix learning rate	Step 50	Step 100	Step 200	Share of backprop's drop
FROM THE 10M-TOKEN CHECKPOINT (VALIDATION LOSS 3.980)				
Backprop, Adam state kept, 0.002	3.933	3.888	3.836	–
Backprop, 0.002	3.940	3.895	3.838	–
Backprop, 0.001	3.891	3.849	3.799	–
Backprop, 0.0005	3.880	3.841	3.796	–
Dust 1k, 0.0015	3.970	3.998	4.048	–
Dust 4k, 0.0015	3.959	3.974	3.986	–
Dust 16k, 0.002	4.006	4.011	4.007	–
Dust 4k, 0.0005	3.900	3.879	3.859	65%
Dust 16k, 0.0005	3.891	3.870	3.846	73%
FROM THE 100M-TOKEN CHECKPOINT (VALIDATION LOSS 3.254)				
Backprop, Adam state kept, 0.002	3.252	3.252	3.249	–
Backprop, 0.002	3.257	3.255	3.250	–
Backprop, 0.001	3.221	3.215	3.204	–
Backprop, 0.0005	3.211	3.206	3.190	–
Backprop, 0.00025	3.213	3.206	3.190	–
Dust 4k, 0.0015	3.242	3.248	3.257	–
Dust 4k, 0.0005	3.224	3.217	3.207	73%
Dust 16k, 0.0005	3.219	3.208	3.199	85%

- At the learning rates tuned for training from scratch, Dust improves neither checkpoint. The loss ends at or above where it started at every population.
- At a learning rate of 0.0005 Dust lowers the loss steadily from both checkpoints, and a larger population lowers it further. It stays above backprop at the same learning rate throughout (Figure 3).
- Backprop also gains from the lower learning rate, and at the 100M-token checkpoint the original rate makes no progress at all. That checkpoint therefore mostly tests whether a noisier gradient estimate still works with a small step, and less whether Dust explores better.

The proposal asked for a checkpoint below a loss of 3 with the GPT-2 tokenizer. Ours start at 3.98 and 3.25 with a 4,096-token vocabulary. Losses are not comparable across tokenizers, and these are certainly weaker models than the one proposed.

Cost

Dust pays for each unit of population with a forward pass. With this release the largest population takes eight GPUs for hours where backprop takes one GPU for seconds or minutes (Table 7). Two things bias the ratios. The times include evaluation, which is most of backprop's wall clock, so the ratios understate the difference in training compute. And the minimal release reruns the whole network for every draw, which the paper's own implementation avoids.

Table 7. Wall-clock cost, evaluation included. The last column is GPU-hours relative to backprop with AdamW at the same token budget.

Run	GPUs	Seconds per step	GPU-hours	Relative to backprop
1M TOKENS, 61 STEPS				
Backprop (AdamW)	1	0.54	0.009	1
Dust (SGD), 256	1	5.3	0.09	10
Dust (SGD), 1k	1	16.9	0.29	31
Dust (SGD), 4k	4	16.8	1.14	125
Dust (SGD), 16k	8	32.1	4.35	478
Dust (Adam), 256	1	6.7	0.11	12
Dust (Adam), 1k	1	22.0	0.37	41
Dust (Adam), 4k	4	21.8	1.48	162
Dust (Adam), 16k	8	42.2	5.73	629
10M TOKENS, 610 STEPS				
Backprop (AdamW)	1	0.14	0.024	1
Dust (SGD), 256	1	4.8	0.82	34
Dust (SGD), 1k	1	16.5	2.79	114
Dust (SGD), 4k	4	16.3	11.03	453
Dust (SGD), 16k	8	31.5	42.69	1,752
Dust (Adam), 256	1	6.2	1.04	43
Dust (Adam), 1k	1	21.5	3.64	149
Dust (Adam), 4k	4	21.3	14.43	592
Dust (Adam), 16k	8	41.6	56.43	2,316

Claims we did not rerun

Three of the paper's claims are outside what the release lets us run. For two of them the paper's own numbers already say something, and we set those out here. Nothing in this section is our measurement.

Larger models. The paper reports that at a fixed population larger models reach a lower loss. Backprop shows the same pattern in the same table, so the comparison that isolates Dust is its distance to backprop at each size. That distance grows with model size at every population from 1k up (Table 8). The study uses SGD, and its largest model is worse than the next smaller one for both methods, which suggests 10M tokens is too little data for it.

Table 8. Test loss of Dust minus that of backprop at the same model size, from the paper's model-size study at 10M tokens (its Figure 4). The last row is backprop's own test loss.

Parameters	2.0M	7.3M	38M	243M
Dust (SGD), 64	0.525	0.490	0.543	0.671
Dust (SGD), 256	0.306	0.296	0.343	0.371
Dust (SGD), 1k	0.085	0.095	0.143	0.166
Dust (SGD), 4k	0.009	0.029	0.038	0.076
Dust (SGD), 16k	-0.009	-0.001	0.021	0.038
Backprop (SGD), test loss	5.180	5.066	5.015	5.048

Gradient alignment. The paper measures the cosine between Dust's estimate and the backprop gradient at populations up to 131,072 and fits $c_{\max}/\sqrt{1+c/K}$ per layer type. Its largest training population is 16,384. Putting that value into the published fits gives a cosine near one half for the feed-forward layers and near one fifth for keys and values (Table 9). The paper's point that the cosine does not fall between 10M and 1B tokens may well hold; our runs show that this alone does not keep the loss level with backprop under Adam.

Table 9. Cosine to the backprop gradient implied by the paper's fitted parameters at its 100M-token checkpoint (its Appendix G), evaluated at the largest training population and at the largest measured one. Our arithmetic on the published parameters.

Layer type	Ceiling	At 16,384	At 131,072
Feed-forward output	0.88	0.51	0.79
Feed-forward input	0.86	0.43	0.73
Attention output	0.87	0.57	0.81
Attention query	1.00	0.42	0.79
Attention key	1.00	0.16	0.42
Attention value	0.88	0.26	0.58
Value gate	0.67	0.34	0.57
Value embedding	1.00	0.18	0.46
Token embedding	0.73	0.49	0.68
Output head	1.00	0.98	1.00

EGGROLL. The weight-space baseline is not in the release, so the claim of a thousand to ten thousand times better efficiency is unchecked. In the paper, the measured part is that EGGROLL at 256 times the population still trails Dust; the larger factor extrapolates a curve that is still falling.

Outside criticism

Three public reactions to the paper, quoted as collected in the [Digg roundup](#) of the discussion, and what our runs say about each ([Table 10](#)).

Table 10. Criticisms and what this reproduction shows.

Who	Point	What our runs show
Sadhika Malladi	A FineWeb validation loss near 5 is not a meaningful amount of pretraining, and plots should use steps or GPU-hours, not tokens.	Supported. On the same 10M tokens backprop with AdamW goes below 4 (Table 2), and the GPU-hour ratios are in Table 7 .
You Jiacheng	If backprop lacks search, continue a backprop-trained checkpoint with the zeroth-order method and see whether it is effective.	Tested on weaker checkpoints than proposed. Dust keeps improving them at a reduced learning rate and stays behind backprop (Table 6).
Kaiyue Wen	Asked whether optimizing the activations with gradient descent could work.	Not tested here.

Limits of this reproduction

- One seed per Dust run, except the cell in [Table 5](#). Backprop's three seeds differ by about 0.06 at 1M tokens and 0.08 at 10M, so differences of that size between Dust cells are not resolved.
- Dust under Adam is a reconstruction. The split of draws across blocks is our assumption; it matches the four published points at 1M tokens and has no published reference at 10M.
- At 10M tokens Dust's Adam settings were varied along two axes at one population only.
- The largest population here is 16,384. The paper's Adam ladder at 1M tokens goes to 65,536, and its claim of ending below backprop rests on an extrapolation beyond that.
- Continuation runs last 200 steps from two checkpoints, with one learning rate below the default for Dust.

References

1. Dahal, Mandal, Gülbahar and Vegesna (2026). [Dust: Pretraining Transformers Without Backpropagation](#). [Code](#).
2. Malladi et al. (2023). [Fine-Tuning Language Models with Just Forward Passes](#).
3. Sarkar et al. (2025). [Evolution Strategies at the Hyperscale](#).
4. Digg (2026). [Dust debuts as a zeroth-order approach to transformer pretraining](#).

Code, the records of every run and a PDF of this report are at github.com/huskydoge/dust, a fork of the official release. It adds `dust_adam.py`, `dust_cpt.py`, `baselines/backprop_adam.py` and `prepare_cpt_data.py`; the official `dust.py` and `baselines/backprop.py` are unchanged.